

1 DT-Classifier- Day - Outlook - Temp - Humidity - Wind - Play

Tennis as Classifier

```
[1]: # Importing necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, confusion_matrix, \
    ConfusionMatrixDisplay
```

```
[2]: # Load the dataset
data = pd.read_csv('DT.csv')
```

```
[4]: data
```

```
[4]:
```

	Day	Outlook	Temp	Humidity	Wind	PlayTennis	Unnamed: 6
0	D1	sunny	hot	high	weak	No	NaN
1	D2	sunny	hot	high	strong	No	NaN
2	D3	overcast	hot	high	weak	Yes	NaN
3	D4	rain	mild	high	weak	Yes	NaN
4	D5	rain	cold	normal	weak	Yes	NaN
5	D6	rain	cold	normal	strong	No	NaN
6	D7	overcast	cold	normal	strong	Yes	NaN
7	D8,	sunny	mild	high	weak	No	NaN
8	D9	sunny	cold	normal	weak	Yes	NaN
9	D10	rain	mild	normal	weak	Yes	NaN
10	D11	sunny	mild	normal	strong	Yes	NaN
11	D12	overcast	mild	high	strong	Yes	NaN
12	D13	overcast	hot	normal	weak	Yes	NaN
13	D14	rain	mild	high	strong	No	NaN
14	D15	sunny	hot	normal	weak	No	NaN

```
[6]: # Encoding categorical variables
label_encoders = {}
for column in data.columns[1:]: # Ignore 'Day' as it's an identifier
    le = LabelEncoder()
```

```
data[column] = le.fit_transform(data[column])
label_encoders[column] = le
```

```
[7]: # Separating features and target variable
X = data[['Outlook', 'Temp', 'Humidity', 'Wind']] # Feature columns
y = data['PlayTennis'] # Target column
```

```
[8]: # Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↳random_state=42)
```

```
[9]: # Train the Decision Tree classifier
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

# Make predictions on the test set
y_pred = model.predict(X_test)
```

```
[10]: y_pred
```

```
[10]: array([1, 1, 0])
```

```
[11]: y_test
```

```
[11]: 9      1
      11     1
      0      0
      Name: PlayTennis, dtype: int32
```

```
[12]: # Calculate and print accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Model accuracy:", accuracy)
```

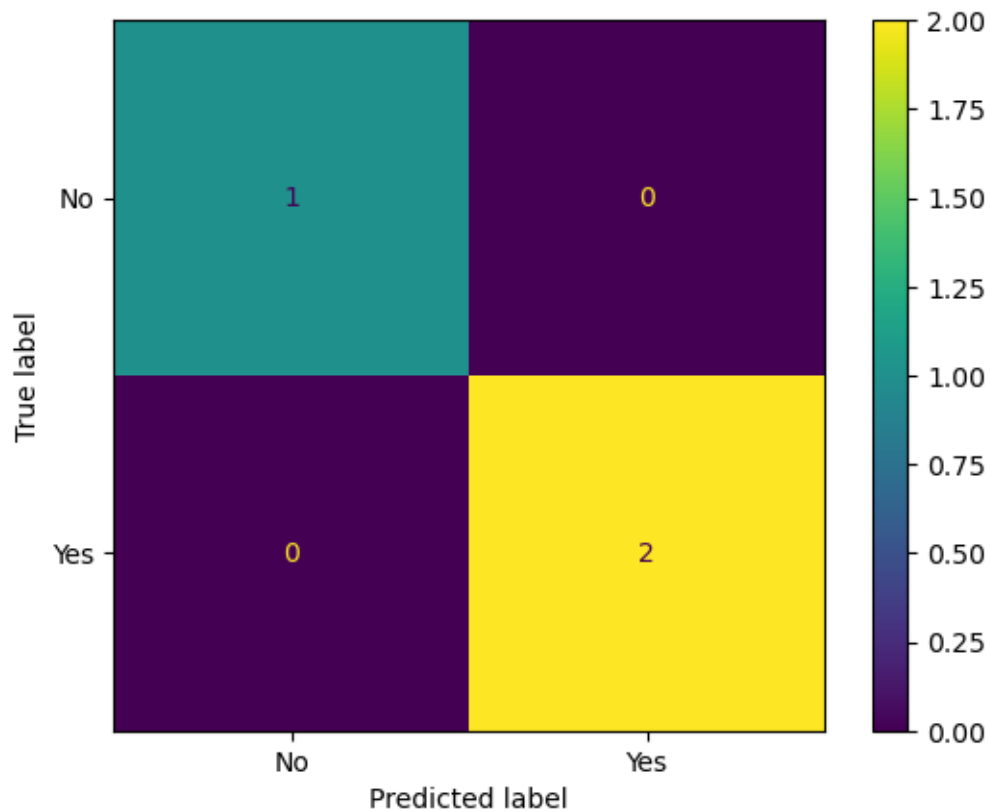
Model accuracy: 1.0

```
[13]: # Display confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(conf_matrix)
ConfusionMatrixDisplay.from_predictions(y_test, y_pred,
↳display_labels=label_encoders['PlayTennis'].classes_)
```

Confusion Matrix:

```
[[1 0]
 [0 2]]
```

[13]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x28921e31cd0>



```
[14]: # Show actual vs predicted values for each test sample
print("\nActual vs Predicted:")
for actual, predicted in zip(y_test, y_pred):
    print(f"Actual: {label_encoders['PlayTennis'].
    ↪inverse_transform([actual])[0]}, "
          f"Predicted: {label_encoders['PlayTennis'].
    ↪inverse_transform([predicted])[0]}")
```

Actual vs Predicted:

Actual: Yes, Predicted: Yes

Actual: Yes, Predicted: Yes

Actual: No, Predicted: No

```
[15]: # Example prediction for new input {rain, mild, high, strong}
input_data = pd.DataFrame({
    'Outlook': [label_encoders['Outlook'].transform(['rain'])[0]],
    'Temp': [label_encoders['Temp'].transform(['mild'])[0]],
    'Humidity': [label_encoders['Humidity'].transform(['high'])[0]],
```

```
    'Wind': [label_encoders['Wind'].transform(['strong'])[0]]  
})
```

```
[16]: # Predict whether to play tennis  
prediction = model.predict(input_data)  
result = label_encoders['PlayTennis'].inverse_transform(prediction)  
print("\nPrediction for input {rain, mild, high, strong}: Play Tennis =",  
      ↪result[0])
```

Prediction for input {rain, mild, high, strong}: Play Tennis = No